Métodos de Análisis de Datos 1

Unidad 2: Exploración y Visualización de Datos

Clases 8 a 13

Objetivos de la unidad

Al finalizar esta unidad, los estudiantes serán capaces de:

- Calcular e interpretar medidas de tendencia central y dispersión
- Analizar la distribución de variables
- Detectar e interpretar outliers
- Realizar análisis de relaciones entre variables
- Crear visualizaciones efectivas con matplotlib, seaborn y ggplot2
- Comunicar hallazgos de forma clara y precisa

1 Clase 8: Estadística Descriptiva Univariada

1.1 Introducción

El análisis exploratorio de datos (EDA - Exploratory Data Analysis) es el proceso de examinar datos para descubrir patrones, detectar anomalías, probar hipótesis y verificar supuestos mediante resúmenes estadísticos y representaciones gráficas.

¿Por qué es importante el EDA?

- Revela la estructura subyacente de los datos
- Identifica variables importantes
- Detecta outliers y anomalías
- Valida supuestos para modelos
- Sugiere transformaciones apropiadas
- Guía la selección de modelos

1.2 Tipos de variables

1.2.1 Variables cuantitativas

- **Discretas:** Valores contables (número de hijos, cantidad de ventas)
- **Continuas:** Cualquier valor en un intervalo (altura, peso, temperatura)

1.2.2 Variables cualitativas (categóricas)

- **Nominales:** Sin orden natural (color, ciudad, género)
- **Ordinales:** Con orden natural (nivel educativo, satisfacción)

1.3 Medidas de tendencia central

Las medidas de tendencia central indican el valor "típico" o "central" de una distribución.

1.3.1 Media aritmética

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

- **Ventajas:** Usa toda la información, única
- **Desventajas:** Sensible a outliers
- **Usar cuando:** Distribución simétrica, sin outliers extremos

Python:

```
media = df['columna'].mean()
```

R:

```
media <- mean(df$columna)
```

1.3.2 Mediana

El valor que divide la distribución en dos mitades iguales. Para datos ordenados $x_1, x_2, \ldots, x_n$:

$$\text{Mediana} = \begin{cases} x_{(n+1)/2} & \text{si } n \text{ es impar} \\ \frac{x_{n/2}+x_{n/2+1}}{2} & \text{si } n \text{ es par} \end{cases}$$

- **Ventajas:** Robusta a outliers
- **Desventajas:** No usa toda la información
- **Usar cuando:** Distribución asimétrica o presencia de outliers

Python:

```
mediana = df['columna'].median()
```

R:

```
mediana <- median(df$columna)
```

1.3.3 Moda

El valor más frecuente en la distribución.

- **Ventajas:** Aplicable a datos cualitativos
- **Desventajas:** Puede no ser única
- **Usar cuando:** Variables categóricas

Python:

```
moda = df['columna'].mode()[0]  # Primera moda si hay m ltiples
```

R:

```r
# Funci n personalizada
moda <- function(x) {
  ux <- unique(x)
  ux[which.max(tabulate(match(x, ux)))]
}
```

Ejemplo: Media vs Mediana

Salarios: \$30k, \$32k, \$35k, \$38k, \$40k, \$200k
Media: \$62.5k (distorsionada por el outlier)
Mediana: \$36.5k (valor más representativo)
En este caso, la mediana es más apropiada.

1.4 Medidas de dispersión

Las medidas de dispersión cuantifican la variabilidad de los datos.

1.4.1 Rango

$$\text{Rango} = \text{Máximo} - \text{Mínimo}$$

- Simple de calcular
- Muy sensible a outliers
- No considera la distribución completa

1.4.2 Rango intercuartílico (IQR)

$$\text{IQR} = Q_3 - Q_1$$

Donde Q_1 es el primer cuartil (percentil 25) y Q_3 es el tercer cuartil (percentil 75).

- Robusto a outliers
- Captura la dispersión del 50% central de los datos

Python:

```python
Q1 = df['columna'].quantile(0.25)
Q3 = df['columna'].quantile(0.75)
IQR = Q3 - Q1
```

R:

```r
IQR_val <- IQR(df$columna)
```

1.4.3 Varianza

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

La varianza mide el promedio de las desviaciones al cuadrado respecto a la media.

- Usa toda la información
- Unidades al cuadrado (difícil interpretación)
- Sensible a outliers

1.4.4 Desviación estándar

$$s = \sqrt{s^2} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

La raíz cuadrada de la varianza, en las mismas unidades que los datos originales.

Python:

```
varianza = df['columna'].var()
desv_std = df['columna'].std()
```

R:

```
varianza <- var(df$columna)
desv_std <- sd(df$columna)
```

Interpretación de la desviación estándar

Si los datos siguen aproximadamente una distribución normal:

- 68% de los datos están dentro de ± 1 desviación estándar de la media
- 95% están dentro de ± 2 desviaciones estándar
- 99.7% están dentro de ± 3 desviaciones estándar

Esta es la **regla empírica** o **regla 68-95-99.7**.

1.4.5 Coeficiente de variación

$$CV = \frac{s}{\bar{x}} \times 100\%$$

Medida de dispersión relativa, útil para comparar la variabilidad de variables con diferentes unidades o escalas.

- Adimensional (porcentaje)
- Permite comparar dispersión entre diferentes variables
- Solo para variables con razón (no para temperaturas en Celsius, por ejemplo)

Python:

```
cv = (df['columna'].std() / df['columna'].mean()) * 100
```

R:

```
cv <- (sd(df$columna) / mean(df$columna)) * 100
```

1.5 Medidas de forma

1.5.1 Asimetría (Skewness)

Mide la falta de simetría de la distribución.

$$\text{Skewness} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{x_i - \bar{x}}{s}\right)^3$$

- **Skewness = 0:** Distribución simétrica
- **Skewness > 0:** Asimetría positiva (cola derecha más larga)
- **Skewness < 0:** Asimetría negativa (cola izquierda más larga)

Python:

```
from scipy import stats
asimetria = stats.skew(df['columna'])
# O con pandas
asimetria = df['columna'].skew()
```

R:

```
library(moments)
asimetria <- skewness(df$columna)
```

1.5.2 Curtosis (Kurtosis)

Mide el "apuntamiento" o concentración de valores alrededor de la media.

$$\text{Kurtosis} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{x_i - \bar{x}}{s}\right)^4 - 3$$

- **Kurtosis = 0:** Mesocúrtica (normal)
- **Kurtosis > 0:** Leptocúrtica (colas pesadas, pico agudo)
- **Kurtosis < 0:** Platicúrtica (colas ligeras, pico plano)

Python:

```
from scipy import stats
curtosis = stats.kurtosis(df['columna'])
# O con pandas
curtosis = df['columna'].kurtosis()
```

R:

```
library(moments)
curtosis <- kurtosis(df$columna) - 3  # Exceso de curtosis
```

2 Clase 9: Análisis de Distribuciones

2.1 Visualización de distribuciones univariadas

2.1.1 Histogramas

Los histogramas muestran la distribución de una variable continua dividiendo el rango en intervalos (bins) y contando cuántas observaciones caen en cada intervalo.

Python:

```
import matplotlib.pyplot as plt

plt.hist(df['columna'], bins=30, edgecolor='black')
plt.xlabel('Variable')
plt.ylabel('Frecuencia')
plt.title('Histograma')
plt.show()
```

R:

```
library(ggplot2)

ggplot(df, aes(x = columna)) +
  geom_histogram(bins = 30, color = "black", fill = "steelblue") +
  labs(x = "Variable", y = "Frecuencia", title = "Histograma")
```

Cuidado con el número de bins

- Muy pocos bins: Se pierde información de la forma
- Demasiados bins: Ruido, difícil interpretación
- **Regla de Sturges:** $k = 1 + \log_2(n)$
- **Regla de Rice:** $k = 2n^{1/3}$
- Lo mejor: Experimentar con diferentes valores

2.1.2 Gráficos de densidad

Estimación suave de la función de densidad de probabilidad.

Python:

```
import seaborn as sns

sns.kdeplot(data=df, x='columna', fill=True)
plt.xlabel('Variable')
plt.ylabel('Densidad')
plt.title('Gr fico de Densidad')
plt.show()
```

R:

```
ggplot(df, aes(x = columna)) +
  geom_density(fill = "steelblue", alpha = 0.5) +
  labs(x = "Variable", y = "Densidad", title = "Gr fico de Densidad")
```

2.1.3 Boxplots (Diagramas de caja)

Resumen visual de los cinco números: mínimo, Q1, mediana, Q3, máximo.

Componentes:

- **Caja:** Rango intercuartílico (IQR)
- **Línea central:** Mediana
- **Bigotes:** Se extienden hasta $1.5 \times$ IQR desde los cuartiles
- **Puntos:** Outliers (fuera de los bigotes)

Python:

```
plt.boxplot(df['columna'])
plt.ylabel('Valor')
plt.title('Boxplot')
plt.show()
```

```
# O con seaborn
sns.boxplot(data=df, y='columna')
plt.show()
```

R:

```
ggplot(df, aes(y = columna)) +
  geom_boxplot(fill = "steelblue") +
  labs(y = "Valor", title = "Boxplot")
```

2.1.4 Violin plots

Combinación de boxplot y gráfico de densidad.

Python:

```
sns.violinplot(data=df, y='columna')
plt.show()
```

R:

```
ggplot(df, aes(y = columna)) +
  geom_violin(fill = "steelblue") +
  labs(y = "Valor", title = "Violin Plot")
```

2.2 Pruebas de normalidad

Muchos métodos estadísticos asumen que los datos siguen una distribución normal.

2.2.1 Q-Q Plot (Quantile-Quantile Plot)

Compara los cuantiles de los datos con los cuantiles teóricos de una distribución normal.

- Si los puntos siguen la línea diagonal, los datos son aproximadamente normales
- Desviaciones en los extremos indican colas pesadas o ligeras

Python:

```
from scipy import stats
import matplotlib.pyplot as plt

stats.probplot(df['columna'], dist="norm", plot=plt)
plt.title('Q-Q Plot')
plt.show()
```

R:

```
qqnorm(df$columna)
qqline(df$columna)
```

2.2.2 Test de Shapiro-Wilk

Prueba estadística de normalidad.

- H_0: Los datos provienen de una distribución normal
- H_1: Los datos no provienen de una distribución normal

- Si $p < 0.05$, rechazar H_0 (datos no normales)

Python:

```
from scipy import stats

statistic, p_value = stats.shapiro(df['columna'])
print(f"Estad stico: {statistic}, p-value: {p_value}")

if p_value > 0.05:
    print("Los datos parecen normales (no rechazamos H0)")
else:
    print("Los datos NO parecen normales (rechazamos H0)")
```

R:

```
resultado <- shapiro.test(df$columna)
print(resultado)
```

Limitaciones del test de Shapiro-Wilk

- Muy sensible con muestras grandes (detecta desviaciones triviales)
- Funciona mejor con muestras de tamaño moderado (50-2000)
- Combinar con inspección visual (Q-Q plot, histograma)

2.3 Transformaciones para normalizar

Cuando los datos no son normales, se pueden aplicar transformaciones:

2.3.1 Transformación logarítmica

$$y = \log(x)$$

- Útil para datos con asimetría positiva
- Requiere $x > 0$
- Si hay ceros: $y = \log(x + 1)$

2.3.2 Transformación de raíz cuadrada

$$y = \sqrt{x}$$

- Menos drástica que log
- Útil para datos de conteo

2.3.3 Transformación Box-Cox

$$y = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \text{si } \lambda \neq 0 \\ \log(x) & \text{si } \lambda = 0 \end{cases}$$

Encuentra automáticamente la mejor transformación potencia.

Python:

```python
from scipy import stats

# Box-Cox (requiere datos positivos)
transformed, lambda_param = stats.boxcox(df['columna'])
print(f"Lambda  ptimo : {lambda_param}")
```

R:

```r
library(MASS)

# Box-Cox
bc <- boxcox(columna ~ 1, data = df)
lambda <- bc$x[which.max(bc$y)]
```

3 Clase 10: Detección de Outliers

3.1 ¿Qué son los outliers?

Los outliers (valores atípicos) son observaciones que se desvían significativamente del resto de los datos.

Tipos de outliers

- **Errores de medición:** Deben corregirse o eliminarse
- **Errores de entrada:** Deben corregirse
- **Valores genuinos pero raros:** Pueden ser los más interesantes (fraudes, casos extremos)
- **Muestreo de otra población:** Considerar análisis separado

3.2 Métodos univariados

3.2.1 Método IQR (Rango Intercuartílico)

Define outliers como valores fuera del rango:

$$[Q_1 - 1.5 \times \text{IQR}, \quad Q_3 + 1.5 \times \text{IQR}]$$

Python:

```python
Q1 = df['columna'].quantile(0.25)
Q3 = df['columna'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df['columna'] < lower_bound) | (df['columna'] >
    upper_bound)]
print(f"Outliers detectados: {len(outliers)}")
```

R:

```r
Q1 <- quantile(df$columna, 0.25)
Q3 <- quantile(df$columna, 0.75)
IQR_val <- Q3 - Q1
```

```r
lower_bound <- Q1 - 1.5 * IQR_val
upper_bound <- Q3 + 1.5 * IQR_val

outliers <- df[df$columna < lower_bound | df$columna > upper_bound, ]
cat("Outliers detectados:", nrow(outliers), "\n")
```

3.2.2 Método Z-score

Define outliers como valores con $|z| > 3$ (o a veces $|z| > 2.5$).

$$z = \frac{x - \bar{x}}{s}$$

Python:

```python
from scipy import stats

z_scores = np.abs(stats.zscore(df['columna']))
outliers = df[z_scores > 3]
print(f"Outliers detectados: {len(outliers)}")
```

R:

```r
z_scores <- abs((df$columna - mean(df$columna)) / sd(df$columna))
outliers <- df[z_scores > 3, ]
cat("Outliers detectados:", nrow(outliers), "\n")
```

Z-score vs IQR

Z-score:

- Asume distribución normal
- Sensible a outliers extremos (la media y std se ven afectadas)

IQR:

- No asume distribución específica
- Robusto a outliers
- Más conservador (detecta menos outliers)

3.2.3 Método de desviación absoluta mediana (MAD)

Más robusto que Z-score:

$$\text{MAD} = \text{mediana}(|x_i - \text{mediana}(x)|)$$

$$\text{MAD-score} = \frac{0.6745(x_i - \text{mediana}(x))}{\text{MAD}}$$

Outliers: $|\text{MAD-score}| > 3.5$

Python:

```python
from scipy import stats

mad = stats.median_abs_deviation(df['columna'])
median = df['columna'].median()
mad_scores = 0.6745 * (df['columna'] - median) / mad

outliers = df[np.abs(mad_scores) > 3.5]
```

R:

```r
mad_val <- mad(df$columna)
median_val <- median(df$columna)
mad_scores <- 0.6745 * (df$columna - median_val) / mad_val

outliers <- df[abs(mad_scores) > 3.5, ]
```

3.3 Métodos multivariados

3.3.1 Distancia de Mahalanobis

Mide la distancia de un punto al centro de la distribución multivariada, considerando la covarianza.

$$D^2 = (x - \mu)^T \Sigma^{-1} (x - \mu)$$

Donde μ es el vector de medias y Σ es la matriz de covarianza.

Python:

```python
from scipy.spatial.distance import mahalanobis

# Seleccionar variables num ricas
X = df[['var1', 'var2', 'var3']].values

# Calcular medias y covarianza
mean = X.mean(axis=0)
cov = np.cov(X.T)
cov_inv = np.linalg.inv(cov)

# Distancias de Mahalanobis
distances = [mahalanobis(x, mean, cov_inv) for x in X]

# Outliers: distancia > umbral (ej: chi-cuadrado con p=0.001)
from scipy.stats import chi2
threshold = chi2.ppf(0.999, df=X.shape[1])
outliers = df[distances > threshold]
```

R:

```r
library(mvoutlier)

# Seleccionar variables
X <- df[, c('var1', 'var2', 'var3')]

# Distancia de Mahalanobis
distances <- mahalanobis(X, colMeans(X), cov(X))

# Outliers
threshold <- qchisq(0.999, df = ncol(X))
outliers <- df[distances > threshold, ]
```

3.3.2 Isolation Forest

Algoritmo de machine learning para detección de anomalías.

Idea: Los outliers son más fáciles de aislar en particiones aleatorias del espacio.

Python:

```
from sklearn.ensemble import IsolationForest

# Entrenar modelo
iso_forest = IsolationForest(contamination=0.1, random_state=42)
predictions = iso_forest.fit_predict(df[['var1', 'var2', 'var3']])

# -1 indica outlier, 1 indica normal
outliers = df[predictions == -1]
print(f"Outliers detectados: {len(outliers)}")
```

R:

```
library(isotree)

# Entrenar modelo
modelo <- isolation.forest(df[, c('var1', 'var2', 'var3')])
scores <- predict(modelo, df[, c('var1', 'var2', 'var3')])

# Outliers: scores altos
threshold <- quantile(scores, 0.9)
outliers <- df[scores > threshold, ]
```

3.4 Tratamiento de outliers

1. **Investigar:** ¿Son errores o valores legítimos?
2. **Documentar:** Registrar qué se encontró y qué se hizo
3. **Opciones:**
   - **Conservar:** Si son valores genuinos de interés
   - **Eliminar:** Si son errores claros (con justificación)
   - **Transformar:** Aplicar transformaciones (log, sqrt)
   - **Winsorizar:** Reemplazar con percentiles (ej: percentil 99)
   - **Análisis separado:** Modelo con y sin outliers
   - **Usar métodos robustos:** Que no se vean afectados por outliers

4 Clase 11: Análisis Bivariado

4.1 Relación entre variables cuantitativas

4.1.1 Scatterplots (gráficos de dispersión)

Visualizan la relación entre dos variables continuas.

Python:

```
plt.scatter(df['var1'], df['var2'], alpha=0.5)
plt.xlabel('Variable 1')
plt.ylabel('Variable 2')
plt.title('Scatterplot')
```

```
plt.show()

# Con seaborn (m s opciones)
sns.scatterplot(data=df, x='var1', y='var2', hue='categoria')
plt.show()
```

R:

```
ggplot(df, aes(x = var1, y = var2)) +
  geom_point(alpha = 0.5) +
  labs(x = "Variable 1", y = "Variable 2", title = "Scatterplot")

# Con color por categor a
ggplot(df, aes(x = var1, y = var2, color = categoria)) +
  geom_point(alpha = 0.5)
```

4.1.2 Correlación lineal (Pearson)

Mide la fuerza y dirección de la relación lineal entre dos variables:

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

- $r \in [-1, 1]$
- $r = 1$: Correlación positiva perfecta
- $r = -1$: Correlación negativa perfecta
- $r = 0$: Sin correlación lineal
- $|r| < 0.3$: Correlación débil
- $0.3 \leq |r| < 0.7$: Correlación moderada
- $|r| \geq 0.7$: Correlación fuerte

Python:

```
correlacion = df['var1'].corr(df['var2'])
print(f"Correlaci n: {correlacion:.3f}")

# Matriz de correlaci n
correlation_matrix = df[['var1', 'var2', 'var3']].corr()
print(correlation_matrix)
```

R:

```
correlacion <- cor(df$var1, df$var2)
cat("Correlaci n:", correlacion, "\n")

# Matriz de correlaci n
correlation_matrix <- cor(df[, c('var1', 'var2', 'var3')])
print(correlation_matrix)
```

Limitaciones de la correlación de Pearson

- Solo mide relaciones **lineales**
- Sensible a outliers
- Asume variables continuas y distribución bivariada normal
- **Correlación no implica causalidad**

4.1.3 Correlación de Spearman

Correlación basada en rangos, más robusta a outliers y no asume linealidad.

Python:

```
from scipy.stats import spearmanr

corr, p_value = spearmanr(df['var1'], df['var2'])
print(f"Correlaci n de Spearman: {corr:.3f}, p-value: {p_value:.4f}")
```

R:

```
resultado <- cor.test(df$var1, df$var2, method = "spearman")
print(resultado)
```

4.1.4 Heatmap de correlación

Visualización de matriz de correlación.

Python:

```
import seaborn as sns

correlation_matrix = df.corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',
            center=0, fmt='.2f', square=True)
plt.title('Matriz de Correlaci n')
plt.show()
```

R:

```
library(corrplot)

correlation_matrix <- cor(df[, sapply(df, is.numeric)])

corrplot(correlation_matrix, method = "color", type = "upper",
         addCoef.col = "black", tl.col = "black", tl.srt = 45)
```

4.2 Relación entre variable categórica y cuantitativa

4.2.1 Boxplots por grupo

Python:

```
sns.boxplot(data=df, x='categoria', y='valor')
plt.title('Distribuci n por Categor a')
plt.show()
```

R:

```
ggplot(df, aes(x = categoria, y = valor, fill = categoria)) +
  geom_boxplot() +
  labs(title = "Distribuci n por Categor a")
```

4.2.2 Violin plots por grupo

Python:

```
sns.violinplot(data=df, x='categoria', y='valor')
plt.show()
```

R:

```
ggplot(df, aes(x = categoria, y = valor, fill = categoria)) +
  geom_violin() +
  labs(title = "Violin Plot por Categor a")
```

4.3 Relación entre variables categóricas

4.3.1 Tablas de contingencia

Python:

```
# Tabla de frecuencias
tabla = pd.crosstab(df['categoria1'], df['categoria2'])
print(tabla)

# Tabla de proporciones
tabla_prop = pd.crosstab(df['categoria1'], df['categoria2'], normalize=
    True)
print(tabla_prop)
```

R:

```
# Tabla de frecuencias
tabla <- table(df$categoria1, df$categoria2)
print(tabla)

# Tabla de proporciones
tabla_prop <- prop.table(tabla)
print(tabla_prop)
```

4.3.2 Test de Chi-cuadrado

Prueba de independencia entre variables categóricas.

- H_0: Las variables son independientes
- H_1: Las variables están asociadas

Python:

```
from scipy.stats import chi2_contingency

tabla = pd.crosstab(df['categoria1'], df['categoria2'])
chi2, p_value, dof, expected = chi2_contingency(tabla)
```

```
print(f"Chi-cuadrado: {chi2:.3f}")
print(f"p-value: {p_value:.4f}")

if p_value < 0.05:
    print("Rechazamos H0: las variables est n asociadas")
else:
    print("No rechazamos H0: las variables parecen independientes")
```

R:

```
resultado <- chisq.test(tabla)
print(resultado)
```

5 Clase 12: Visualización con Python

5.1 Matplotlib: Fundamentos

Matplotlib es la librería base para visualización en Python.

5.1.1 Anatomía de una figura

- **Figure:** El contenedor principal
- **Axes:** El área donde se dibuja (puede haber múltiples)
- **Axis:** Los ejes x e y
- **Artist:** Elementos visuales (líneas, texto, etc.)

Estructura básica:

```
import matplotlib.pyplot as plt

# M todo pyplot (simple)
plt.plot([1, 2, 3], [1, 4, 9])
plt.xlabel('X')
plt.ylabel('Y')
plt.title('T tulo')
plt.show()

# M todo orientado a objetos (m s control)
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot([1, 2, 3], [1, 4, 9])
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_title('T tulo')
plt.show()
```

5.1.2 Múltiples subplots

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# axes es un array 2D
axes[0, 0].plot(x1, y1)
axes[0, 0].set_title('Gr fico 1')

axes[0, 1].scatter(x2, y2)
```

```
axes[0, 1].set_title('Gr fico 2')

axes[1, 0].hist(data, bins=30)
axes[1, 0].set_title('Gr fico 3')

axes[1, 1].boxplot(data)
axes[1, 1].set_title('Gr fico 4')

plt.tight_layout()
plt.show()
```

5.1.3 Personalización

```
fig, ax = plt.subplots(figsize=(10, 6))

# L nea
ax.plot(x, y,
        color='steelblue',     # Color
        linewidth=2,            # Grosor
        linestyle='--',         # Estilo: '-', '--', '-.', ':'
        marker='o',             # Marcador
        markersize=8,           # Tama o marcador
        alpha=0.7,              # Transparencia
        label='Datos')          # Etiqueta para leyenda

# Configuraci n de ejes
ax.set_xlabel('X', fontsize=12)
ax.set_ylabel('Y', fontsize=12)
ax.set_title('T tulo', fontsize=14, fontweight='bold')

# L mites
ax.set_xlim([0, 10])
ax.set_ylim([0, 100])

# Grilla
ax.grid(True, alpha=0.3)

# Leyenda
ax.legend(loc='best')

# Guardar
plt.savefig('grafico.png', dpi=300, bbox_inches='tight')
plt.show()
```

5.2 Seaborn: Visualización estadística

Seaborn está construido sobre matplotlib y facilita la creación de gráficos estadísticos atractivos.

5.2.1 Configuración de estilo

```
import seaborn as sns

# Estilos disponibles: darkgrid, whitegrid, dark, white, ticks
sns.set_style("whitegrid")

# Contextos: paper, notebook, talk, poster
```

```
sns.set_context("notebook")

# Paletas de colores
sns.set_palette("Set2")
```

5.2.2 Gráficos de distribución

```
# Histograma con KDE
sns.histplot(data=df, x='columna', kde=True, bins=30)

# Distribuci n por categor a
sns.histplot(data=df, x='valor', hue='categoria', kde=True)

# KDE plot
sns.kdeplot(data=df, x='columna', fill=True)

# Distribuci n conjunta
sns.jointplot(data=df, x='var1', y='var2', kind='scatter')
# kind: 'scatter', 'kde', 'hist', 'hex', 'reg'
```

5.2.3 Gráficos categóricos

```
# Boxplot
sns.boxplot(data=df, x='categoria', y='valor')

# Violin plot
sns.violinplot(data=df, x='categoria', y='valor')

# Swarm plot (puntos sin superposici n)
sns.swarmplot(data=df, x='categoria', y='valor')

# Bar plot (con intervalo de confianza)
sns.barplot(data=df, x='categoria', y='valor')

# Count plot (frecuencias)
sns.countplot(data=df, x='categoria')
```

5.2.4 Gráficos de relación

```
# Scatterplot
sns.scatterplot(data=df, x='var1', y='var2', hue='categoria',
                size='var3', alpha=0.6)

# Line plot
sns.lineplot(data=df, x='tiempo', y='valor', hue='grupo')

# Regression plot
sns.regplot(data=df, x='var1', y='var2')

# LM plot (con facetas)
sns.lmplot(data=df, x='var1', y='var2', hue='categoria',
           col='otra_categoria')
```

5.2.5 Pair plot

Matriz de scatterplots para múltiples variables.

```
sns.pairplot(df, hue='categoria', diag_kind='kde')
plt.show()
```

5.2.6 Heatmap

```
# Matriz de correlaci n
corr = df.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', center=0,
            fmt='.2f', square=True, linewidths=1)
plt.title('Matriz de Correlaci n')
plt.show()
```

5.2.7 FacetGrid: Gráficos múltiples

```
# Grid por categor as
g = sns.FacetGrid(df, col='categoria1', row='categoria2', height=4)
g.map(sns.scatterplot, 'var1', 'var2')
g.add_legend()
plt.show()
```

6 Clase 13: Visualización con R (ggplot2)

6.1 Gramática de gráficos

ggplot2 implementa la "gramática de gráficos" de Leland Wilkinson:

- **Data:** Los datos a visualizar
- **Aesthetics (aes):** Mapeo de variables a propiedades visuales (x, y, color, size, etc.)
- **Geometries (geom):** Tipo de gráfico (puntos, líneas, barras, etc.)
- **Facets:** Paneles múltiples
- **Statistics:** Transformaciones estadísticas
- **Coordinates:** Sistema de coordenadas
- **Themes:** Apariencia general

6.2 Estructura básica

```
library(ggplot2)

ggplot(data = df, aes(x = var1, y = var2)) +
  geom_point() +
  labs(x = "Variable 1", y = "Variable 2", title = "T tulo")
```

El operador + agrega capas al gráfico.

6.3 Aesthetics (aes)

```
ggplot(df, aes(x = var1, y = var2,
               color = categoria,     # Color por categor a
               size = var3,           # Tama o por variable
               shape = grupo,         # Forma por grupo
               alpha = 0.6)) +        # Transparencia
  geom_point()
```

6.4 Geometrías principales

6.4.1 Scatterplots

```
# B sico
ggplot(df, aes(x = var1, y = var2)) +
  geom_point()

# Con color y tama o
ggplot(df, aes(x = var1, y = var2, color = categoria, size = var3)) +
  geom_point(alpha = 0.6) +
  scale_color_brewer(palette = "Set2")
```

6.4.2 Line plots

```
ggplot(df, aes(x = tiempo, y = valor, color = grupo)) +
  geom_line(size = 1) +
  geom_point(size = 2)
```

6.4.3 Histogramas

```
# B sico
ggplot(df, aes(x = columna)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "black")

# Por categor a
ggplot(df, aes(x = valor, fill = categoria)) +
  geom_histogram(bins = 30, alpha = 0.6, position = "identity")
```

6.4.4 Densidad

```
ggplot(df, aes(x = columna, fill = categoria)) +
  geom_density(alpha = 0.5)
```

6.4.5 Boxplots

```
ggplot(df, aes(x = categoria, y = valor, fill = categoria)) +
  geom_boxplot() +
  theme(legend.position = "none")
```

6.4.6 Violin plots

```
ggplot(df, aes(x = categoria, y = valor, fill = categoria)) +
  geom_violin() +
  geom_boxplot(width = 0.1, fill = "white")  # Boxplot superpuesto
```

6.4.7 Bar plots

```
# Frecuencias
ggplot(df, aes(x = categoria)) +
  geom_bar(fill = "steelblue")

# Valores agregados
ggplot(df, aes(x = categoria, y = valor)) +
  geom_bar(stat = "summary", fun = "mean", fill = "steelblue") +
  labs(y = "Media de Valor")
```

6.5 Facetas

```
# Facetas por columna
ggplot(df, aes(x = var1, y = var2)) +
  geom_point() +
  facet_wrap(~ categoria)

# Facetas en grilla
ggplot(df, aes(x = var1, y = var2)) +
  geom_point() +
  facet_grid(categoria1 ~ categoria2)
```

6.6 Estadísticas

```
# L nea de regresi n
ggplot(df, aes(x = var1, y = var2)) +
  geom_point() +
  geom_smooth(method = "lm", se = TRUE)  # se = intervalo de confianza

# Smooth (LOESS)
ggplot(df, aes(x = var1, y = var2)) +
  geom_point() +
  geom_smooth(se = TRUE)
```

6.7 Escalas

```
# Escala de colores
ggplot(df, aes(x = var1, y = var2, color = valor)) +
  geom_point() +
  scale_color_gradient(low = "blue", high = "red")

# Escala logar tmica
ggplot(df, aes(x = var1, y = var2)) +
  geom_point() +
  scale_x_log10() +
  scale_y_log10()
```

```
# L mites de ejes
ggplot(df, aes(x = var1, y = var2)) +
  geom_point() +
  xlim(0, 100) +
  ylim(0, 50)
```

6.8 Temas

```
# Temas predefinidos
ggplot(df, aes(x = var1, y = var2)) +
  geom_point() +
  theme_minimal()
# Otros: theme_bw(), theme_classic(), theme_dark(), theme_void()

# Personalizaci n
ggplot(df, aes(x = var1, y = var2)) +
  geom_point() +
  theme(
    plot.title = element_text(size = 16, face = "bold"),
    axis.title = element_text(size = 12),
    axis.text = element_text(size = 10),
    legend.position = "bottom",
    panel.grid.major = element_line(color = "gray80"),
    panel.grid.minor = element_blank()
  )
```

6.9 Guardar gráficos

```
# Crear gr fico
p <- ggplot(df, aes(x = var1, y = var2)) +
  geom_point()

# Guardar
ggsave("grafico.png", plot = p, width = 10, height = 6, dpi = 300)
# Formatos: png, pdf, svg, jpg
```

6.10 Extensiones de ggplot2

- **ggpubr:** Gráficos listos para publicación
- **patchwork:** Combinar múltiples gráficos
- **gganimate:** Animaciones
- **plotly + ggplotly:** Gráficos interactivos
- **ggridges:** Ridge plots
- **GGally:** Pair plots y más

```
# Ejemplo con GGally
library(GGally)

ggpairs(df, columns = c('var1', 'var2', 'var3'),
        mapping = aes(color = categoria))
```

Resumen de la Unidad 2

- La **estadística descriptiva** resume datos con medidas de tendencia central (media, mediana, moda) y dispersión (rango, IQR, varianza, desviación estándar)
- El **análisis de distribuciones** usa histogramas, densidades, boxplots y Q-Q plots
- Las **pruebas de normalidad** (Shapiro-Wilk) validan supuestos
- Los **outliers** se detectan con métodos IQR, Z-score, MAD, Mahalanobis, Isolation Forest
- El **análisis bivariado** explora relaciones con scatterplots, correlación (Pearson, Spearman), tablas de contingencia
- **Matplotlib** es la librería base de Python; **seaborn** facilita gráficos estadísticos
- **ggplot2** implementa la gramática de gráficos en R con sintaxis consistente y elegante

Conceptos clave

- Media, mediana, moda
- Varianza, desviación estándar, IQR
- Asimetría (skewness), curtosis (kurtosis)
- Histogramas, densidad, boxplots, violin plots
- Q-Q plots, test de Shapiro-Wilk
- Transformaciones: log, sqrt, Box-Cox
- Outliers: IQR, Z-score, MAD, Mahalanobis
- Scatterplots, correlación (Pearson, Spearman)
- Tablas de contingencia, chi-cuadrado
- Matplotlib: figure, axes, subplots
- Seaborn: histplot, boxplot, scatterplot, heatmap
- ggplot2: aes, geom, facet, theme

Próxima unidad

En la Unidad 3 nos enfocaremos en **Modelado Estadístico**: regresión lineal simple y múltiple, regularización (Ridge, LASSO), validación cruzada y evaluación de modelos.